

PART III — BUILDING THE RETRIEVAL PIPELINE

Chapter 12

Advanced Retrieval Techniques

“Every technique in this chapter was researched, understood, and left for the version of this project that needs it.”

Chapter 12 — Advanced Retrieval Techniques

Memora's own research log is unusually candid about this gap. Its eighth research topic, "Retrieval Ranking Algorithms," evaluated Maximal Marginal Relevance, BM25 hybrid search, cross-encoder reranking, and Reciprocal Rank Fusion against the shipped cosine-only pipeline, wrote down a recommended upgrade order, and closed with one honest sentence: not yet implemented. The techniques in this chapter are not hypothetical — they were investigated for this exact retriever — they simply were not the project's next priority. A few of them, like multi-query rephrasing, did ship, just in the later agentic pipeline (Chapter 19B) rather than in the retriever itself.

12.1 The upgrade ladder

Treat the four researched techniques as an ordered ladder rather than four independent options. Each rung keeps everything below it working and adds one specific capability cosine similarity lacks on its own; none of them replace the retriever built in Chapter 11 — they sit on top of it.

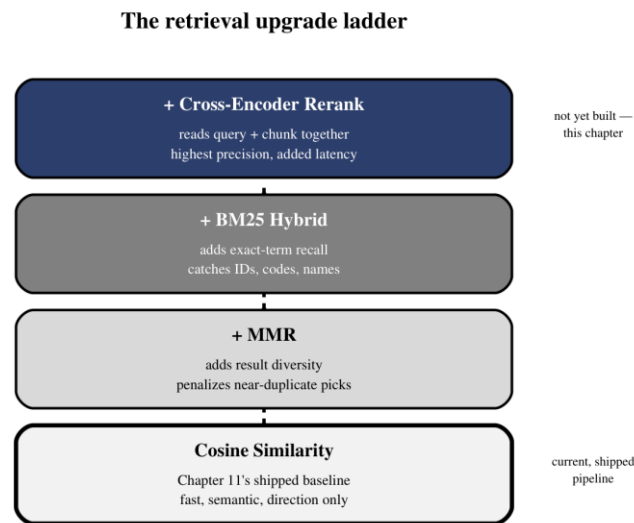


Figure 12.1 — Each rung fixes one specific weakness of plain cosine similarity.

Figure 12.1 mirrors Memora's own recommended order: add MMR first because it costs nothing extra to retrieve (it only changes which already-fetched candidates get kept), add BM25 hybrid scoring next because it fixes an entire class of query cosine embeddings handle poorly, and add cross-encoder reranking last because it is the most accurate rung and the most expensive one, best spent on a small top-10 candidate set rather than every chunk in the collection.

Rung	Fixes	Approximate cost
MMR	Near-duplicate chunks crowding out coverage	Negligible — reorders results already in hand
BM25 hybrid	Exact terms (IDs, model names, codes) dense vectors miss	One extra lexical index + a fusion step
Cross-encoder rerank	Ranking precision on the final short list	One model call per candidate chunk, on a small top-10

12.2 Hybrid search — dense and sparse combined

Definition — BM25

A keyword-frequency ranking function (Best Matching 25) that scores a document against a query using term frequency and inverse document frequency, normalized by document length. It has no notion of meaning — it matches surface tokens.

Dense embeddings and BM25 fail in complementary ways. A cosine search over `all-MiniLM-L6-v2` embeddings is excellent at "documents about the same topic as this question" and poor at exact identifiers — a model name, a part number, an error code — because those tokens carry little of the semantic signal the embedding space was trained to capture. BM25 is the mirror image: it excels at exact-term matching and has no idea that "lowers electricity costs" and "minimizes energy expenses" mean the same thing.

```
from rank_bm25 import BM25Okapi

class BM25Index:
    def __init__(self, chunks: list[dict]):
        self.chunks = chunks
        tokenized = [c["content"].lower().split() for c in chunks]
        self.index = BM25Okapi(tokenized)

    def search(self, query: str, top_k: int) -> list[dict]:
        scores = self.index.get_scores(query.lower().split())
        ranked = sorted(zip(self.chunks, scores), key=lambda p: p[1],
reverse=True)
        return [{"chunk", "bm25_score": score} for chunk, score in
ranked[:top_k]]
```

Analogy — Two ways to search a library

A dense search is asking a librarian who has read every book to point you toward the right shelf. A sparse search is scanning the index card catalog for your exact search term. A librarian who has read everything may forget an exact part number; a card catalog does not know that two books use different words for the same idea. A hybrid search asks both and keeps whichever answer actually helps.

Combining them requires two ranked lists and a way to merge them — which is exactly what Section 12.3 builds.

12.3 Reciprocal Rank Fusion — merging ranked lists

Definition — Reciprocal Rank Fusion (RRF)

A rank-merging formula that scores each document by summing $1 / (k + \text{rank})$ across every ranked list it appears in, where k is a small constant (commonly 60). Documents ranked highly in multiple lists rise to the top of the fused ranking without needing the lists' raw scores to be on comparable scales.

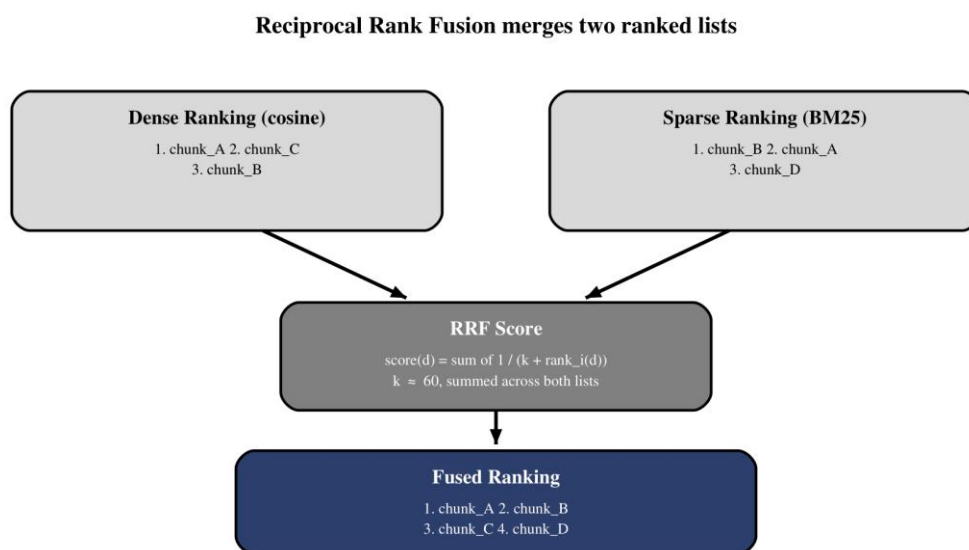


Figure 12.2 — RRF needs only rank position from each list, not comparable score scales.

RRF's real advantage, visible in Figure 12.2, is that it sidesteps score calibration entirely. A cosine similarity and a BM25 score live on completely different numeric scales, and picking a weighted average between them requires tuning a mixing coefficient per corpus. RRF only asks each list for a rank position — first, second, third — so a chunk that both searches consider strong rises to the top regardless of how the two underlying scores were computed.

```
def rrf_fuse(ranked_lists: list[list[str]], k: int = 60) -> list[str]:
    scores: dict[str, float] = {}
    for ranked in ranked_lists:
        for position, doc_id in enumerate(ranked, start=1):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + position)
    return sorted(scores, key=scores.get, reverse=True)
```

It is worth naming a distinction Memora's research notes blur slightly: RRF fuses two rankings over the *same* corpus produced by *different retrieval methods* — dense and sparse search over the same documents collection. That is a different problem from `retrieve_separate`'s decision (Chapter 11.5) to keep the `documents` and `learned_qa` collections as two independent lists rather than one merged ranking. Those two collections differ in trust level and provenance, not in retrieval method — merging them with RRF would blur a distinction the two-track architecture exists specifically to preserve.

12.4 Maximal Marginal Relevance for diversity

Definition — Maximal Marginal Relevance (MMR)

A re-selection strategy that picks each next result by balancing relevance to the query against dissimilarity to results already chosen, controlled by a weight λ between 0 (pure diversity) and 1 (pure relevance).

Plain top-k cosine ranking has a blind spot: if a corpus contains five paraphrased sentences of the same fact, cosine similarity happily returns all five as the top five results, because each one really is highly similar to the query. The retrieved set is technically correct and practically useless — five slots spent saying one thing. MMR fixes this by re-scoring each **candidate** result against what has already been selected, not just against the query.

```
def mmr_select(query_vec, candidates: list[dict], k: int, lambda_mult:
float = 0.5):
    selected: list[dict] = []
    pool = list(candidates)
    while pool and len(selected) < k:
        def mmr_score(c):
            relevance = cosine(query_vec, c["embedding"])
            if not selected:
                return relevance
            redundancy = max(cosine(c["embedding"], s["embedding"]) for s
in selected)
            return lambda_mult * relevance - (1 - lambda_mult) *
redundancy

        best = max(pool, key=mmr_score)
        selected.append(best)
        pool.remove(best)
    return selected
```

MMR needs no new index and no new retrieval call — it re-ranks a candidate pool already fetched with a slightly larger `top_k` than the final answer needs, which is why it is the cheapest rung on the ladder and the one Memora's research flagged as the best immediate upgrade.

12.5 Reranking with cross-encoders

Definition — Cross-encoder

A model that scores relevance by encoding the query and a candidate chunk together, in a single forward pass, rather than encoding each independently and comparing vectors afterward. It cannot be precomputed or indexed — every query/candidate pair requires its own inference call.

Chapter 11's bi-encoder embeds the query and every chunk independently, then compares vectors with cosine similarity — fast, and precomputable for every chunk at ingestion time. A cross-encoder such as `'cross-encoder/ms-marco-MiniLM-L-6-v2'` gives up that precomputation for accuracy: it reads the query and one candidate chunk together as a single input and outputs a direct relevance score, letting the model's attention mechanism compare them token by token instead of collapsing each into an independent point in space first.

Property	Bi-encoder (Chapter 11)	Cross-encoder (this section)
Encodes	Query and chunk separately	Query and chunk together
Precomputable	Yes — chunks embedded once at ingestion	No — one inference per query/chunk pair
Cost per query	One query embedding + a vector search	One model call per candidate reranked
Best used for	Narrowing millions of chunks to dozens	Narrowing dozens of chunks to a final few

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

def rerank(query: str, candidates: list[dict], top_k: int) -> list[dict]:
    pairs = [(query, c["content"]) for c in candidates]
    scores = reranker.predict(pairs)
    ranked = sorted(zip(candidates, scores), key=lambda p: p[1],
reverse=True)
    return [{**c, "rerank_score": s} for c, s in ranked[:top_k]]
```

Common pitfall — Reranking the whole collection

A cross-encoder call is orders of magnitude slower than a vector comparison because nothing about it can be precomputed. Never run it over an entire collection — retrieve a wider candidate set with the cheap bi-encoder first (top-10 to top-20), then spend the cross-encoder budget narrowing that small set down to the top-5 that actually reach the prompt.

12.6 Metadata filtering and hybrid filters

Every chunk Chapter 11's retriever returns already carries metadata — `source`, `page`, and (from Chapter 7's chunking pipeline) a `chunk\_seq` position within its source document. ChromaDB accepts a `where` clause alongside the vector query, letting a caller narrow the search space before similarity ranking even runs, rather than filtering a ranked list after the fact.

```
collection.query(
    query_embeddings=[query_embedding.tolist()],
    n_results=top_k,
    where={"source": "policy_manual.pdf"},
    include=["documents", "metadatas", "distances"],
)
```

A hybrid filter narrows twice: once structurally (only this source, only pages after 10, only chunks tagged as a table) and once semantically (the actual vector search among the survivors). Reach for a metadata filter whenever a caller already knows something deterministic about where the answer lives — a filter costs nothing in retrieval quality and everything in retrieval precision, because it removes candidates the similarity score would otherwise have to out-rank on its own.

12.7 Multi-query retrieval — rephrasing for coverage

Definition — Multi-query retrieval

Generating several differently-worded variants of a single user question and retrieving for each, so that a corpus's varied vocabulary is covered by more than one angle of attack, then combining or deduplicating the results before they reach the prompt.

This rung is the one exception in the chapter: Memora actually built it. `app\_workflow/nodes/query\_variants.py` generates several query rephrasings per question and retrieves for each — but an LLM asked for three rephrasings will sometimes hand back three near-identical ones ("motor delays in autism" and "motor skill delays in ASD"), and running retrieval for near-duplicates wastes an entire retrieval-and-validation cycle on a query that was never going to surface anything new. The fix is a two-phase filter, run before any retrieval call is spent: deduplicate near-identical variants by pairwise cosine similarity at a conservative `PRE\_RETRIEVAL\_SIM\_THRESHOLD = 0.95`, then rank the survivors by similarity to the *original* query and keep only as many as an adaptive budget allows.

```
def pre_retrieval_filter(variants: list[str], query_embedding, budget:
int) -> list[str]:
    embeddings = embed_all(variants)
    survivors: list[tuple[str, "vector"]] = []
    for variant, vec in zip(variants, embeddings):
        if any(cosine(vec, s_vec) >= PRE_RETRIEVAL_SIM_THRESHOLD for _,
s_vec in survivors):
            continue # near-duplicate of an already-kept variant
        survivors.append((variant, vec))

    survivors.sort(key=lambda pair: cosine(pair[1], query_embedding),
reverse=True)
    return [variant for variant, _ in survivors[:budget]]
```

The order matters: deduplicating before ranking ensures a high-quality unique variant is never discarded just because a near-duplicate happened to rank slightly higher by chance. Cosine was chosen over cheaper token-overlap (Jaccard) similarity deliberately — Jaccard fails on exactly the paraphrase pairs this filter exists to catch, since two paraphrases can share almost no vocabulary at all.

12.8 HyDE — hypothetical document embeddings

Definition — HyDE (Hypothetical Document Embeddings)

A retrieval strategy that first asks an LLM to write a plausible, hypothetical answer to the query, then embeds and searches with that generated passage instead of the raw question — on the theory that an answer-shaped passage sits closer in embedding space to real answer-shaped chunks than a short question does.

HyDE was on the table early in Memora's own architecture decision — the choice between a fixed classic-RAG pipeline and an agentic loop that decides its own retrieval strategy explicitly listed HyDE as a considered alternative before the project committed to the agentic decision loop covered starting in Chapter 15. It was not rejected as flawed; it was set aside because an iterative, self-correcting agent loop solved the same underlying problem — a short question retrieving poorly against long-form source text — more generally, by letting the model reformulate and retry rather than betting everything on one generated hypothetical passage.

```
def hyde_retrieve(query: str, llm, retriever, top_k: int):
    hypothetical = llm_invoke(
        llm, [{"role": "user", "content": f"Write a short passage
answering: {query}"}],
        caller_tag="HYDE",
    ).content
    return retriever.retrieve(hypothetical, top_k=top_k)
```

HyDE trades one extra LLM call for a better-shaped query vector. That trade is worth making when questions are short and source material is long-form prose — exactly the mismatch it is designed to close — and worth skipping when an agent can already retry with reformulated queries, as Section 12.7's multi-query filter and Chapter 18's agent loop both do.

12.9 Parent-child and small-to-big retrieval

Definition — Parent-child retrieval

Indexing and searching small, precise child chunks for matching accuracy, but returning each match's larger parent chunk — or its neighboring siblings — to the language model, so retrieval precision and context completeness are optimized independently instead of trading off against a single chunk size.

Small chunks embed precisely — a tight paragraph produces a focused vector — but a language model reading only that paragraph often loses surrounding context that would have prevented a misreading. Large chunks preserve context but dilute the embedding, since a long chunk's vector is an average over many different ideas, some relevant and some not. Parent-child retrieval is one answer: search small, return big.

Memora never built this pattern, but it solved a closely related problem with a different mechanism. Chapter 22B's Neighbor-Aware Compression (NAC) merges consecutive chunks from the same source, identified by their shared `'chunk_seq'` metadata, *after* retrieval rather than restructuring the index beforehand — restoring the document flow that fixed-size chunking breaks, without maintaining two chunk granularities side by side. Parent-child retrieval solves the boundary problem before search; NAC solves it after. Either is defensible — the tradeoff is index complexity (parent-child) against a compression pass on the critical path (NAC).

12.10 Contextual compression — extracting only relevant sentences

Definition — Contextual compression

Reducing a retrieved chunk to only the sentences that bear on the current query, discarding the rest of the chunk's content before it reaches the prompt, so a fixed context budget carries a higher density of query-relevant material.

A retrieved chunk is relevant as a whole — it passed the similarity threshold — but rarely uniformly relevant sentence by sentence; a paragraph about a policy typically contains one governing clause and several sentences of surrounding boilerplate. Contextual compression trims that surrounding text after retrieval, before the prompt is assembled.

Memora's research evaluated this rung most thoroughly of all ten in this chapter, and — unlike the others — actually shipped it, as the extractive stage of a three-part compression hierarchy documented in full in Chapter 22B. Extractive compression, using SentenceTransformers cosine similarity between each sentence and the query, was judged the safest option: no generation, no hallucination risk, purely a subtractive filter over sentences that were already retrieved as faithful source text. LangChain's `'ContextualCompressionRetriever'` plus `'LLMChainExtractor'` was evaluated as a framework-level alternative and set aside for the same reason recurring throughout this chapter — an extra dependency and latency cost for a capability the project could implement directly, with a clearer view of exactly what it was doing to the text.

- Score each sentence in a retrieved chunk against the query independently.
- Keep sentences above a relevance threshold; drop the rest.
- Never rewrite a kept sentence — compression here is subtractive, not generative.
- Guard against over-compression: if retention falls below a safety floor, prefer the original chunk.

Ten rungs, one honest throughline: a retriever is never finished, only appropriately sized for what it currently needs to do. Chapter 11's cosine-only mechanism is correct and sufficient for a small, well-scoped corpus; every technique in this chapter earns its cost only once a real failure mode — missed exact terms, redundant results, imprecise ranking, an over-long chunk — actually shows up in practice. With ranked, filtered, sufficiently precise evidence in hand by whichever rung a project needs, the next job is turning that evidence and the original question into an answer a reader can trust.